

MP1: A Tiny SNS System Design Document

Phani Jyothi Kurada

UIN: 335006950

CSCE 662

September 30, 2025

1 Introduction

This document describes the system-level design of the Tiny Social Network Service (SNS) MP1 Machine Problem Implementation. It outlines the architecture, components, client-server interaction, and test cases demonstrating functionality with screenshots attached wherever applicable.

2 Environment Setup

The SNS system was developed and executed inside a Docker container to ensure a consistent environment. A preconfigured image `liuyidockers/csce438_env:latest` was used. The container `csce438_container` was started using the Docker Desktop engine, which manages the container runtime.

Once the container was running, the interactive terminal was accessed with the following command:

```
$docker exec -it csce438_container bash
```

This opened a shell inside the running container, from which the server and client programs were compiled and executed.

3 System Components

The system architecture is composed of two primary components: the **Server** and the **Client**. Their interaction occurs through two main modes of operation: **Command mode**, which handles requests such as login, list, follow, and unfollow, and **Timeline mode**, which manages the posting and streaming of messages.

3.1 Server Side Design

The server is implemented in `tsd.cc` using gRPC for communication with clients. It maintains the state of all connected users, their timelines, and Follow / UnFollow relationships.

3.1.1 Client Data Structure

Each client is represented by the following structure:

```
struct Client {
    std::string username;
    bool connected = true;
    std::vector<Client*> client_followers;
    std::vector<Client*> client_following;
    ServerReaderWriter<Message, Message>* stream = nullptr;
    std::vector<Message> timeline;
    std::unordered_map<std::string, time_t> following_since;
};
```

The server maintains a global list of all clients:

```
std::vector<Client*> client_db;
```

3.1.2 Client Lookup Function

One of the essential components of the server implementation is the `findClient` function. This function is responsible for searching through the in-memory client database to locate a specific client object based on the provided username. It ensures that various server operations (such as login, follow, unfollow, or timeline retrieval) can quickly retrieve the corresponding `Client` structure.

The function performs a linear search through the global `client_db` vector, which stores pointers to all connected clients. If a match is found, it returns a pointer to the corresponding `Client` instance. If no match is found, the function returns `nullptr`, indicating that the user does not exist in the system.

The implementation of this function is shown below:

```
Client* findClient(const std::string& username) {
    for (auto* c : client_db) {
        if (c->username == username)
            return c;
    }
    return nullptr;
}
```

This lookup is fundamental for server operations. For example, it is invoked during the `Login` RPC to check if a user already exists, during the `FOLLOW` and `UNFOLLOW` operations to locate the involved clients, and within the `Timeline` RPC to identify the active user.

3.1.3 RPC Implementations

Login : The `Login` RPC checks if a user exists in `client_db`. If not, it creates a new client. A client cannot login twice simultaneously. On success, the server sets `connected = true`.

```
Status Login(ServerContext* context, const Request* request, Reply*
    reply) override {
```

```

std::string uname = request->username();
Client* existing = findClient(uname);
if (!existing) {
    Client* new_client = new Client();
    new_client->username = uname;
    new_client->connected = true;
    client_db.push_back(new_client);
} else {
    if (existing->connected) {
        reply->set_msg("Login failed: User already logged in");
        return Status(grpc::StatusCode::ALREADY_EXISTS, "User
            already logged in");
    }
    existing->connected = true;
}
reply->set_msg("Login successful");
return Status::OK;
}

```

List : The List RPC returns:

- All users registered on the server.
- The followers of the requesting client.

```

Status List(ServerContext* context, const Request* request,
    ListReply* list_reply) override {
    Client* current = findClient(request->username());
    if (!current) return Status(grpc::StatusCode::NOT_FOUND, "User
        not found");

    for (auto* c : client_db) {
        list_reply->add_all_users(c->username);
    }

    for (auto* f : current->client_followers) {
        list_reply->add_followers(f->username);
    }
    return Status::OK;
}

```

Follow : The Follow RPC allows a client to follow another user:

- Updates both `client_following` and `client_followers` vectors.
- Records the current timestamp in `following_since` to mark when the follow started.

```

Status Follow(ServerContext* context, const Request* request, Reply*
reply) override {
    std::string follower_name = request->username();
    std::string target_name = request->arguments(0);

    Client* follower = findClient(follower_name);
    Client* target = findClient(target_name);

    if (!follower || !target) {
        reply->set_msg("User not found");
        return Status::OK;
    }
    if (follower == target) {
        reply->set_msg("Cannot follow yourself");
        return Status::OK;
    }

    for (auto* f : follower->client_following) {
        if (f == target) {
            reply->set_msg("Already following");
            return Status::OK;
        }
    }

    follower->client_following.push_back(target);
    target->client_followers.push_back(follower);

    follower->following_since[target->username] = time(NULL);

    reply->set_msg("Followed successfully");
    return Status::OK;
}

```

UnFollow : The UnFollow RPC removes a follow relationship:

- Removes target from `client_following` and follower from `client_followers`.
- Deletes the corresponding entry from `following_since`.

```

Status UnFollow(ServerContext* context, const Request* request,
Reply* reply) override {
    std::string follower_name = request->username();
    std::string target_name = request->arguments(0);

    Client* follower = findClient(follower_name);
    Client* target = findClient(target_name);

    if (!follower || !target) {

```

```

        reply->set_msg("User not found");
        return Status::OK;
    }
    if (follower == target) {
        reply->set_msg("Cannot unfollow yourself");
        return Status::OK;
    }

    auto it = std::find(follower->client_following.begin(), follower
->client_following.end(), target);
    if (it == follower->client_following.end()) {
        reply->set_msg("Not following this user");
        return Status::OK;
    }

    follower->client_following.erase(it);
    follower->following_since.erase(target->username);

    auto it2 = std::remove_if(target->client_followers.begin(),
        target->client_followers.end(),
        [&](Client* c) { return c == follower;
        });
    target->client_followers.erase(it2, target->client_followers.end
());

    reply->set_msg("Unfollowed successfully");
    return Status::OK;
}

```

Timeline : The Timeline RPC establishes a bidirectional streaming session:

- The server retrieves the last 20 posts from followed users **after the time the client started following each user**.
- Messages are sorted in **descending timestamp order** so that the newest messages appear first.
- Each new post made by the client is broadcast immediately to all their followers who are currently connected.

```

Status Timeline(ServerContext* context, ServerReaderWriter<Message,
Message>* stream) override {
    Client* current_user = nullptr;

    Message init_msg;
    if (!stream->Read(&init_msg)) {
        return Status(grpc::StatusCode::INVALID_ARGUMENT, "No
        username received");
    }
}

```

```

}

current_user = findClient(init_msg.username());
if (!current_user) {
    return Status(grpc::StatusCode::NOT_FOUND, "User not found")
        ;
}

current_user->stream = stream;

std::vector<Message> recent_posts;
for (auto* followed : current_user->client_following) {
    auto follow_time = current_user->following_since[followed->
        username];
    for (auto& m : followed->timeline) {
        if (m.timestamp().seconds() >= follow_time) {
            recent_posts.push_back(m);
        }
    }
}

std::sort(recent_posts.begin(), recent_posts.end(),
    [](const Message& a, const Message& b) {
        return a.timestamp().seconds() > b.timestamp().
            seconds();
    });

if (recent_posts.size() > 20) {
    recent_posts.resize(20);
}

for (auto& m : recent_posts) {
    stream->Write(m);
}

Message client_msg;
while (stream->Read(&client_msg)) {
    current_user->timeline.push_back(client_msg);
    for (auto* follower : current_user->client_followers) {
        if (follower->stream) {
            follower->stream->Write(client_msg);
        }
    }
}

current_user->stream = nullptr;
return Status::OK;

```

```
}
```

3.1.4 Server Execution

The `main` function is the entry point of the server program. It parses the command-line argument for the server port, initializes Google logging, and then starts the gRPC server by calling `RunServer(port)`, which begins listening for client requests. The server listens on a configurable port (default 3010) and registers the `SNSServiceImpl` service:

```
void RunServer(std::string port_no) {
    std::string server_address = "0.0.0.0:" + port_no;
    SNSServiceImpl service;

    ServerBuilder builder;
    builder.AddListeningPort(server_address, grpc::
        InsecureServerCredentials());
    builder.RegisterService(&service);
    std::unique_ptr<Server> server(builder.BuildAndStart());
    std::cout << "Server listening on " << server_address << std::
        endl;
    log(INFO, "Server listening on " + server_address);

    server->Wait();
}
```

The following commands are used to compile and run the server inside the Docker container:

```
docker exec -it csce438_container bash
cd /home/csce438/mp1_skeleton
make clean
make -j4
./tsd -p 3010
```

```

PS C:\Users\kurad> docker exec -it csce438_container bash
csce438@cf7e955cca43:~$ cd /home/csce438/mp1_skeleton
csce438@cf7e955cca43:~/mp1_skeleton$ make clean
rm -f *.o *.pb.cc *.pb.h tsc tsd
csce438@cf7e955cca43:~/mp1_skeleton$ make -j4
protoc -I . --cpp_out=. sns.proto
protoc -I . --grpc_out=. --plugin=protoc-gen-grpc='which grpc_cpp_plugin' sn
s.proto
g++ -std=c++17 'pkg-config --cflags protobuf grpc absl_flags absl_flags_pars
e' -c -o tsd.o tsd.cc
g++ -std=c++17 'pkg-config --cflags protobuf grpc absl_flags absl_flags_pars
e' -c -o client.o client.cc
g++ -std=c++17 'pkg-config --cflags protobuf grpc absl_flags absl_flags_pars
e' -c -o tsc.o tsc.cc
g++ -std=c++17 'pkg-config --cflags protobuf grpc absl_flags absl_flags_pars
e' -c -o sns.pb.o sns.pb.cc
g++ -std=c++17 'pkg-config --cflags protobuf grpc absl_flags absl_flags_pars
e' -c -o sns.grpc.pb.o sns.grpc.pb.cc
g++ sns.pb.o sns.grpc.pb.o tsd.o -L/usr/local/lib 'pkg-config --libs --stati
c protobuf grpc++ absl_flags absl_flags_parse absl_absl_check absl_absl_log
absl_algorithm absl_base absl_bind_front absl_bits absl_btree absl_cleanup a
bsl_cord absl_core_headers absl_debugging absl_die_if_null absl_dynamic_anno
tations absl_flags absl_flat_hash_map absl_flat_hash_set absl_function_ref a
bsl_hash absl_layout absl_log_initialize absl_log_severity absl_memory absl_
node_hash_map absl_node_hash_set absl_optional absl_span absl_status absl_st
atusor absl_strings absl_synchronization absl_time absl_type_traits absl_utili
lity absl_utility absl_variant' -lutf8_validation -pthread -Wl,--no-as-needed -lgrpc++_refl
ection -Wl,--as-needed -ldl -lglog -g -o tsd
g++ client.o sns.pb.o sns.grpc.pb.o tsc.o -L/usr/local/lib 'pkg-config --lib
s --static protobuf grpc++ absl_flags absl_flags_parse absl_absl_check absl_
absl_log absl_algorithm absl_base absl_bind_front absl_bits absl_btree absl_
cleanup absl_cord absl_core_headers absl_debugging absl_die_if_null absl_dyn
amic_annotations absl_flags absl_flat_hash_map absl_flat_hash_set absl_func
tion_ref absl_hash absl_layout absl_log_initialize absl_log_severity absl_mem
ory absl_node_hash_map absl_node_hash_set absl_optional absl_span absl_statu
s absl_statusor absl_strings absl_synchronization absl_time absl_type_traits
absl_utility absl_utility absl_variant' -lutf8_validation -pthread -Wl,--no-as-needed -lgr
pc++_reflection -Wl,--as-needed -ldl -lglog -g -o tsc
csce438@cf7e955cca43:~/mp1_skeleton$ ./tsd -p 3010
Server listening on 0.0.0.0:3010

```

Figure 1: Server Execution

The screenshot above shows the Server has been started in the docker container and is listening on port 3010.

3.2 Client Side Design

The client component of the SNS system is responsible for connecting to the server, sending commands, receiving timeline messages asynchronously, and displaying them to the user. The key features are:

- Connects to the server via gRPC using the hostname and port provided.
- Sends commands to the server: **LOGIN**, **LIST**, **FOLLOW**, **UNFOLLOW**, **TIMELINE**.
- Receives timeline messages asynchronously from followed users.
- Displays the last 20 messages in reverse chronological order (newest first) from followed users, respecting the `following_since` timestamp.

3.2.1 Message Creation

The function `MakeMessage` creates a new `Message` object with the specified username and message content. It also attaches a timestamp representing the current time using Google Protobuf's `Timestamp`.


```

Message MakeMessage(const std::string &username, const std::string &
msg) {
    Message m;
    m.set_username(username);
    m.set_msg(msg);
    google::protobuf::Timestamp *timestamp = new google::protobuf::
        Timestamp();
    timestamp->set_seconds(time(NULL));
    timestamp->set_nanos(0);
    m.set_allocated_timestamp(timestamp);
    return m;
}

```

3.2.2 Client Class Design

The `Client` class implements the client-side functionality and communicates with the server via gRPC. The class has the following responsibilities:

- Maintaining connection parameters: hostname, username, and port.
- Handling commands sent by the user: LOGIN, LIST, FOLLOW, UNFOLLOW, TIME-LINE.
- Interfacing with the gRPC stub to perform RPCs.
- Receiving and displaying timeline messages asynchronously.

```

class Client : public IClient {
public:
    Client(const std::string &hname, const std::string &uname, const
std::string &p)
        : hostname(hname), username(uname), port(p) {}

protected:
    virtual int connectTo();                % Connects to the server and
        logs in
    virtual IReply processCommand(std::string &input); % Parses and
        executes commands
    virtual void processTimeline();          % Starts timeline streaming

private:
    std::string hostname;                   % Server hostname
    std::string username;                   % Client username
    std::string port;                       % Server port
    std::unique_ptr<SNSService::Stub> stub_; % gRPC stub

    IReply Login();                         % Performs LOGIN RPC

```

```

IReply List();                                % Performs LIST RPC
IReply Follow(const std::string &username);    % Performs FOLLOW
RPC
IReply UnFollow(const std::string &username); % Performs
UNFOLLOW RPC
void Timeline(const std::string &username);    % Handles TIMELINE
streaming
};

```

Each private method corresponds to a server RPC, encapsulating the communication logic and returning a structured response (IReply) to the client for further processing.

3.2.3 Connecting to the Server

The client connects to the server using gRPC and attempts to log in with a provided username:

```

// Connect to server and login
int Client::connectTo() {
    std::string addr = hostname + ":" + port;
    stub_ = SNSService::NewStub(grpc::CreateChannel(addr, grpc::
        InsecureChannelCredentials()));
    IReply login_reply = Login();
    if (!login_reply.grpc_status.ok() || login_reply.comm_status !=
        SUCCESS) {
        std::cerr << "Login failed: " << login_reply.grpc_status.
            error_message() << std::endl;
        return -1;
    }
    std::cout << "Login successful as " << username << std::endl;
    return 1;
}

```

The following commands are used to create a new user by connecting to server and log in:

```

docker exec -it csce438_container bash
cd /home/csce438/mp1_skeleton
make clean
make -j4
./tsc -h localhost -p 3010 -u (username)

```



3.2.5 Client RPC Methods

The client class implements methods corresponding to each server RPC. These methods communicate with the server using gRPC and return an `IReply` object containing both gRPC status and command-specific response.

List RPC : The `List` method sends a `Request` with the client's username to the server and retrieves all registered users and the list of followers for the current user. The returned data is stored in vectors rather than printing directly.

```
IReply Client::List() {
    IReply ire;
    Request req;
    req.set_username(username);
    ClientContext context;
    ListReply list_reply;
    Status status = stub_->List(&context, req, &list_reply);
    ire.grpc_status = status;

    if (status.ok()) {
        ire.comm_status = SUCCESS;

        // Fill the vectors instead of printing
        for (int i = 0; i < list_reply.all_users_size(); i++)
            ire.all_users.push_back(list_reply.all_users(i));

        for (int i = 0; i < list_reply.followers_size(); i++)
            ire.followers.push_back(list_reply.followers(i));

    } else {
        ire.comm_status = FAILURE_UNKNOWN;
    }

    return ire;
}
```

Follow RPC : The `Follow` method sends a request to follow another user. Based on the server's response message, the method sets the appropriate status in the `IReply` object.

```
IReply Client::Follow(const std::string &username2) {
    IReply ire;
    ClientContext context;
    Request req;
    Reply rep;

    req.set_username(username);
    req.add_arguments(username2);

    Status status = stub_->Follow(&context, req, &rep);
```

```

    ire.grpc_status = status;

    std::string msg = rep.msg();

    if (msg == "Followed successfully") {
        ire.comm_status = SUCCESS;
    } else if (msg == "Already following") {
        ire.comm_status = FAILURE_ALREADY_EXISTS;
    } else if (msg == "Cannot follow yourself") {
        ire.comm_status = FAILURE_ALREADY_EXISTS;
    } else if (msg == "User not found") {
        ire.comm_status = FAILURE_INVALID_USERNAME;
    } else {
        ire.comm_status = FAILURE_UNKNOWN;
    }

    return ire;
}

```

UnFollow RPC : The UnFollow method works similarly to Follow, sending a request to stop following a user and interpreting the server's response.

```

IReply Client::UnFollow(const std::string &username2) {
    IReply ire;
    ClientContext context;
    Request req;
    Reply rep;

    req.set_username(username);
    req.add_arguments(username2);

    Status status = stub_->UnFollow(&context, req, &rep);
    ire.grpc_status = status;

    std::string msg = rep.msg();

    if (msg == "Unfollowed successfully") {
        ire.comm_status = SUCCESS;
    } else if (msg == "Not following this user") {
        ire.comm_status = FAILURE_NOT_A_FOLLOWER;
    } else if (msg == "Cannot unfollow yourself") {
        ire.comm_status = FAILURE_INVALID_USERNAME;
    } else if (msg == "User not found") {
        ire.comm_status = FAILURE_INVALID_USERNAME;
    } else {
        ire.comm_status = FAILURE_UNKNOWN;
    }
}

```

```

    return ire;
}

```

Login RPC : The Login method performs the login RPC by sending the username to the server and sets the IReply status based on the server's response.

```

IReply Client::Login() {
    IReply ire;
    Request req;
    req.set_username(username);

    ClientContext context;
    Reply reply;
    Status status = stub_->Login(&context, req, &reply);
    ire.grpc_status = status;

    if (status.ok() && reply.msg() == "Login successful") {
        ire.comm_status = SUCCESS;
    } else if (reply.msg() == "Username already exists") {
        ire.comm_status = FAILURE_ALREADY_EXISTS;
        std::cerr << reply.msg() << std::endl; // print server
            message
    } else {
        ire.comm_status = FAILURE_UNKNOWN;
    }
    return ire;
}

```

Timeline Streaming : The Timeline method establishes a bi-directional streaming gRPC channel with the server. It immediately sends the client's username for identification.

A separate reader thread continuously listens for messages from followed users and displays them in real time. The main thread allows the client to send new posts asynchronously. This ensures that timeline updates are received and displayed as soon as they arrive.

```

void Client::Timeline(const std::string &username) {
    ClientContext context;
    std::shared_ptr<ClientReaderWriter<Message, Message>> stream(stub_->Timeline(&context));

    Message init_msg;
    init_msg.set_username(username);
    stream->Write(init_msg);

    std::thread reader([stream]() {
        Message server_msg;
        while (stream->Read(&server_msg)) {
            std::time_t post_time = server_msg.timestamp().seconds();
            displayPostMessage(server_msg.username(), server_msg.msg()
                , post_time);
        }
    });
}

```

```

    }
});

while (true) {
    std::string text = getPostMessage();
    if (text.empty()) continue;

    Message m = MakeMessage(username, text);
    if (!stream->Write(m)) break;
}

stream->WritesDone();
reader.join();
}

```

3.2.6 Main Function

The client initializes the connection parameters and starts the client instance:

```

int main(int argc, char **argv) {
    std::string hostname = "localhost";
    std::string username = "default";
    std::string port = "3010";

    int opt = 0;
    while ((opt = getopt(argc, argv, "h:u:p:")) != -1) {
        switch (opt) {
            case 'h': hostname = optarg; break;
            case 'u': username = optarg; break;
            case 'p': port = optarg; break;
            default: std::cout << "Invalid Command Line Argument\n";
        }
    }

    Client myc(hostname, username, port);
    myc.run();
    return 0;
}

```

3.3 Client-Server Interaction

1. Client logs in → Server validates username and marks the client as connected.
2. Client executes commands → Server processes and returns appropriate responses.
3. Timeline streaming:
 - Last 20 messages (newest first) from followed users after follow timestamp.

- New messages from followed users appear in real-time.

4 Test Case Scenarios

To verify the correctness and robustness of the implemented social network system, several test cases were executed. Each scenario focuses on a specific functionality of the client-server interaction and ensures that the system behaves as expected.

4.1 Test Case 1: Duplicate Login Verification

Objective: Verify that a new user can register with the server and login successfully only once.

Procedure: Launch the client with a new username. The server should create a new user profile and establish a connection. Open another terminal and login using the same username.

Expected Result: The client logs in successfully and receives a confirmation message from the server for the first time, during second time the login must fail.

[illegible]

4.2 Test Case 2: Listing and Following All Users

Objective: Ensure that the client can request and receive a list of all registered users from the server and follow registered users successfully.

Procedure: After logging in as 3 different users, execute the **LIST** command and **FOLLOW** command.(Performed commands according to the given testcase document).

Expected Result: The server returns a list of all users currently registered in the system. Successfull execution of **FOLLOW** commands.

[illegible]

4.3 Test Case 3: Following / UnFollowing a User

Objective: Verify the ability of users to follow and unfollow each other.

Procedure: Execute the FOLLOW <username> command to follow another registered user and UNFOLLOW <username> for unfollowing the followed users. Tested for three different users.

Expected Result: The server updates the follower/following lists, and a confirmation message is displayed on the client.

```

csc438@c7e955cca43: ~/m x csc438@c7e955cca43: ~/m x csc438@c7e955cca43: ~/m x csc438@c7e955cca43: ~/m x
csc438@c7e955cca43:~/m$ ./mp1_skeleton$ ./tsc -h localhost -p 3010 -u u1
Logging Initialized. Client starting...
Login successful as u1

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> FOLLOW u2
Command completed successfully
Cmd> FOLLOW u3
Command completed successfully
Cmd> UNFOLLOW u1
Command failed with invalid username
Cmd> UNFOLLOW u2
Command completed successfully
Cmd> UNFOLLOW u3
Command completed successfully
Cmd> UNFOLLOW u4
Command failed with invalid username
Cmd> |

csc438@c7e955cca43:~/m x csc438@c7e955cca43:~/m x csc438@c7e955cca43:~/m x csc438@c7e955cca43:~/m x
csc438@c7e955cca43:~/m$ ./mp1_skeleton$ ./tsc -h localhost -p 3010 -u u2
Logging Initialized. Client starting...
Login successful as u2

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> FOLLOW u1
Command completed successfully
Cmd> FOLLOW u3
Command completed successfully
Cmd> UNFOLLOW u1
Command completed successfully
Cmd> UNFOLLOW u2
Command failed with invalid username
Cmd> UNFOLLOW u3
Command completed successfully
Cmd> UNFOLLOW u4
Command failed with invalid username
Cmd> |

csc438@c7e955cca43:~/m x csc438@c7e955cca43:~/m x csc438@c7e955cca43:~/m x csc438@c7e955cca43:~/m x
csc438@c7e955cca43:~/m$ ./mp1_skeleton$ ./tsc -h localhost -p 3010 -u u3
Logging Initialized. Client starting...
Login successful as u3

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> FOLLOW u1
Command completed successfully
Cmd> FOLLOW u2
Command completed successfully
Cmd> UNFOLLOW u1
Command completed successfully
Cmd> UNFOLLOW u2
Command failed with invalid username
Cmd> UNFOLLOW u3
Command failed with invalid username
Cmd> UNFOLLOW u4
Command failed with invalid username
Cmd> |

```

4.4 Test Case 4: Combination of Follow / UnFollow / LIST for 3 users

Objective: Ensure that a user can unfollow someone they previously followed and listing the followers that the user has at that moment.

Procedure: Executed the FOLLOW <username>, UNFOLLOW <username> and LIST commands according to testcase document.

Expected Result: The target user is removed from the following list, and a confirmation message is displayed.

```
csc438@c7e955ca43:~/mp x csc438@c7e955ca43:~/mp x csc438@c7e955ca43:~/mp x csc438@c7e955ca43:~/mp x
csc438@c7e955ca43:~/mp_skeleton$ ./tsc -h localhost -p 3010 -u u1
Logging initialized. Client starting...
Login successful as u1

===== TINY SMS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> FOLLOW u2
Command completed successfully
Cmd> FOLLOW u3
Command completed successfully
Cmd> UNFOLLOW u1
Command failed with invalid username
Cmd> LIST
Command completed successfully
All users: u1, u2, u3,
Followers:
Cmd> UNFOLLOW u2
Command completed successfully
Cmd> UNFOLLOW u3
Command completed successfully
Cmd> UNFOLLOW u4
Command failed with invalid username
Cmd> LIST
Command completed successfully
All users: u1, u2, u3,
Followers:
Cmd> LIST
Command completed successfully
All users: u1, u2, u3,
Followers: u2,
Cmd> LIST
Command completed successfully
All users: u1, u2, u3,
Followers:
Cmd> LIST
Command completed successfully
All users: u1, u2, u3,
Followers: u3,
Cmd> LIST
Command completed successfully
All users: u1, u2, u3,
Followers:
Cmd> |

csc438@c7e955ca43:~/mp x csc438@c7e955ca43:~/mp x csc438@c7e955ca43:~/mp x csc438@c7e955ca43:~/mp x
csc438@c7e955ca43:~/mp_skeleton$ ./tsc -h localhost -p 3010 -u u2
Logging initialized. Client starting...
Login successful as u2

===== TINY SMS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> LIST
Command completed successfully
All users: u1, u2, u3,
Followers: u1,
Cmd> LIST
Command completed successfully
All users: u1, u2, u3,
Followers:
Cmd> FOLLOW u1
Command completed successfully
Cmd> FOLLOW u3
Command completed successfully
Cmd> UNFOLLOW u1
Command completed successfully
Cmd> UNFOLLOW u2
Command failed with invalid username
Cmd> LIST
Command completed successfully
All users: u1, u2, u3,
Followers:
Cmd> UNFOLLOW u3
Command completed successfully
Cmd> UNFOLLOW u4
Command failed with invalid username
Cmd> LIST
Command completed successfully
All users: u1, u2, u3,
Followers:
Cmd> LIST
Command completed successfully
All users: u1, u2, u3,
Followers: u3,
Cmd> LIST
Command completed successfully
All users: u1, u2, u3,
Followers:
Cmd> |

csc438@c7e955ca43:~/mp x csc438@c7e955ca43:~/mp x csc438@c7e955ca43:~/mp x csc438@c7e955ca43:~/mp x
csc438@c7e955ca43:~/mp_skeleton$ ./tsc -h localhost -p 3010 -u u3
Logging initialized. Client starting...
Login successful as u3

===== TINY SMS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> LIST
Command completed successfully
All users: u1, u2, u3,
Followers: u1,
Cmd> LIST
Command completed successfully
All users: u1, u2, u3,
Followers:
Cmd> LIST
Command completed successfully
All users: u1, u2, u3,
Followers: u2,
Cmd> LIST
Command completed successfully
All users: u1, u2, u3,
Followers:
Cmd> FOLLOW u1
Command completed successfully
Cmd> FOLLOW u2
Command completed successfully
Cmd> UNFOLLOW u1
Command completed successfully
Cmd> UNFOLLOW u2
Command failed with invalid username
Cmd> LIST
Command completed successfully
All users: u1, u2, u3,
Followers:
Cmd> UNFOLLOW u4
Command failed with invalid username
Cmd> LIST
Command completed successfully
All users: u1, u2, u3,
Followers:
Cmd> |
```

4.5 Test Case 5: Posting a Message to Timeline

Objective: Test the timeline functionality by posting messages between 3 users following each other. The messages should be posted from the point where the user started following and display the last 20 messages in descending order of them being sent.

Procedure: Use the **TIMELINE** command to enter Timeline mode and post a message.

Expected Result: The message should be stored in the user's timeline and propagated to all followers.

```
csce438@cf7e955cca43:~/mp$ ./tsc -h localhost -p 3010 -u u1
Logging Initialized. Client starting...
Login successful as u1

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> FOLLOW u1
Command completed successfully
Cmd> TIMELINE
Command completed successfully
Now you are in the timeline
p1
p2
p3
p4
p5
p6
p7
p8
p9
p10
p11
p12
p13
p14
p15
p16
p17
p18
p19
p20
p21
p22

csce438@cf7e955cca43:~/mp$ ./tsc -h localhost -p 3010 -u u2
Logging Initialized. Client starting...
Login successful as u2

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> FOLLOW u1
Command completed successfully
Cmd> TIMELINE
Command completed successfully
Now you are in the timeline
u1 (Tue Sep 30 00:37:59 2025) >> p22
u1 (Tue Sep 30 00:37:58 2025) >> p21
u1 (Tue Sep 30 00:37:56 2025) >> p20
u1 (Tue Sep 30 00:37:53 2025) >> p19
u1 (Tue Sep 30 00:37:52 2025) >> p18
u1 (Tue Sep 30 00:37:50 2025) >> p17
u1 (Tue Sep 30 00:37:48 2025) >> p16
u1 (Tue Sep 30 00:37:47 2025) >> p15
u1 (Tue Sep 30 00:37:45 2025) >> p14
u1 (Tue Sep 30 00:37:43 2025) >> p13
u1 (Tue Sep 30 00:37:42 2025) >> p12
u1 (Tue Sep 30 00:37:40 2025) >> p11
u1 (Tue Sep 30 00:37:17 2025) >> p10
u1 (Tue Sep 30 00:37:15 2025) >> p9
u1 (Tue Sep 30 00:37:13 2025) >> p8
u1 (Tue Sep 30 00:37:12 2025) >> p7
u1 (Tue Sep 30 00:37:10 2025) >> p6
u1 (Tue Sep 30 00:37:07 2025) >> p5
u1 (Tue Sep 30 00:37:06 2025) >> p4
u1 (Tue Sep 30 00:37:05 2025) >> p3

csce438@cf7e955cca43:~/mp$ ./tsc -h localhost -p 3010 -u u3
Logging Initialized. Client starting...
Login successful as u3

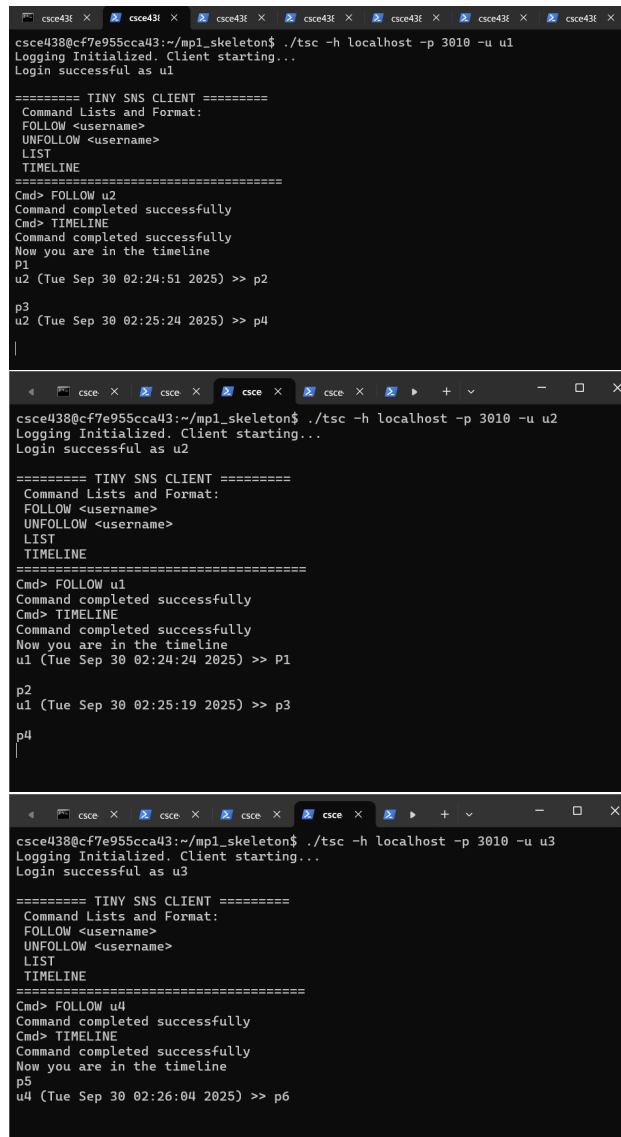
===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> FOLLOW u1
Command completed successfully
Cmd> TIMELINE
Command completed successfully
Now you are in the timeline
u1 (Tue Sep 30 00:37:59 2025) >> p22
u1 (Tue Sep 30 00:37:58 2025) >> p21
u1 (Tue Sep 30 00:37:56 2025) >> p20
u1 (Tue Sep 30 00:37:53 2025) >> p19
u1 (Tue Sep 30 00:37:52 2025) >> p18
u1 (Tue Sep 30 00:37:50 2025) >> p17
u1 (Tue Sep 30 00:37:48 2025) >> p16
u1 (Tue Sep 30 00:37:47 2025) >> p15
u1 (Tue Sep 30 00:37:45 2025) >> p14
u1 (Tue Sep 30 00:37:43 2025) >> p13
u1 (Tue Sep 30 00:37:42 2025) >> p12
u1 (Tue Sep 30 00:37:40 2025) >> p11
```

4.6 Test Case 6: Concurrent User Sessions

Objective: Validate system stability under concurrent access.

Procedure: Launch multiple clients(10) and perform various operations simultaneously (e.g., login, follow, timeline updates).

Expected Result: The server handles all requests reliably without errors or crashes.



The image displays three terminal windows, each representing a different user session (u1, u2, and u3) of the 'TINY SNS CLIENT'. Each window shows the client's command-line interface, including login status, available commands (FOLLOW, UNFOLLOW, LIST, TIMELINE), and the execution of these commands. The windows are arranged vertically, illustrating concurrent access to the system.

```
csce438@cf7e955cca43:~/mp1_skeleton$ ./tsc -h localhost -p 3010 -u u1
Logging Initialized. Client starting...
Login successful as u1

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> FOLLOW u2
Command completed successfully
Cmd> TIMELINE
Command completed successfully
Now you are in the timeline
p1
u2 (Tue Sep 30 02:24:51 2025) >> p2

p3
u2 (Tue Sep 30 02:25:24 2025) >> p4
|

csce438@cf7e955cca43:~/mp1_skeleton$ ./tsc -h localhost -p 3010 -u u2
Logging Initialized. Client starting...
Login successful as u2

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> FOLLOW u1
Command completed successfully
Cmd> TIMELINE
Command completed successfully
Now you are in the timeline
u1 (Tue Sep 30 02:24:24 2025) >> p1

p2
u1 (Tue Sep 30 02:25:19 2025) >> p3

p4
|

csce438@cf7e955cca43:~/mp1_skeleton$ ./tsc -h localhost -p 3010 -u u3
Logging Initialized. Client starting...
Login successful as u3

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> FOLLOW u4
Command completed successfully
Cmd> TIMELINE
Command completed successfully
Now you are in the timeline
p5
u4 (Tue Sep 30 02:26:04 2025) >> p6
```

```
csce438@cf7e955cca43:~/mp1_skeleton$ ./tsc -h localhost -p 3010 -u u4
Logging Initialized. Client starting...
Login successful as u4

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> FOLLOW u5
Command completed successfully
Cmd> TIMELINE
Command completed successfully
Now you are in the timeline
p6
u5 (Tue Sep 30 02:26:09 2025) >> p7

csce438@cf7e955cca43:~/mp1_skeleton$ ./tsc -h localhost -p 3010 -u u5
Logging Initialized. Client starting...
Login successful as u5

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> FOLLOW u3
Command completed successfully
Cmd> TIMELINE
Command completed successfully
Now you are in the timeline
u3 (Tue Sep 30 02:25:47 2025) >> p5

p7

csce438@cf7e955cca43:~/mp1_skeleton$ ./tsc -h localhost -p 3010 -u u6
Logging Initialized. Client starting...
Login successful as u6

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> FOLLOW u7
Command completed successfully
Cmd> FOLLOW u8
Command completed successfully
Cmd> FOLLOW u9
Command completed successfully
Cmd> FOLLOW u10
Command completed successfully
Cmd> TIMELINE
Command completed successfully
Now you are in the timeline
p8
u7 (Tue Sep 30 02:26:51 2025) >> p9
u8 (Tue Sep 30 02:27:16 2025) >> p10
u9 (Tue Sep 30 02:27:20 2025) >> p11
u10 (Tue Sep 30 02:27:26 2025) >> p12
```

```
csce438@cf7e955cca43:~/mp1_skeleton$ ./tsc -h localhost -p 3010 -u u7
Logging Initialized. Client starting...
Login successful as u7

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> FOLLOW u6
Command completed successfully
Cmd> FOLLOW u8
Command completed successfully
Cmd> FOLLOW u9
Command completed successfully
Cmd> FOLLOW u10
Command completed successfully
Cmd> TIMELINE
Command completed successfully
Now you are in the timeline
u6 (Tue Sep 30 02:26:45 2025) >> p8

p9
u8 (Tue Sep 30 02:27:16 2025) >> p10
u9 (Tue Sep 30 02:27:20 2025) >> p11
u10 (Tue Sep 30 02:27:26 2025) >> p12

csce438@cf7e955cca43:~/mp1_skeleton$ ./tsc -h localhost -p 3010 -u u8
Logging Initialized. Client starting...
Login successful as u8

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> FOLLOW u7
Command completed successfully
Cmd> FOLLOW u6
Command completed successfully
Cmd> FOLLOW u9
Command completed successfully
Cmd> FOLLOW u10
Command completed successfully
Cmd> TIMELINE
Command completed successfully
Now you are in the timeline
u6 (Tue Sep 30 02:26:45 2025) >> p8

u7 (Tue Sep 30 02:26:51 2025) >> p9

p10
u9 (Tue Sep 30 02:27:20 2025) >> p11
u10 (Tue Sep 30 02:27:26 2025) >> p12
|

csce438@cf7e955cca43:~/mp1_skeleton$ ./tsc -h localhost -p 3010 -u u9
Logging Initialized. Client starting...
Login successful as u9

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> FOLLOW u7
Command completed successfully
Cmd> FOLLOW u8
Command completed successfully
Cmd> FOLLOW u6
Command completed successfully
Cmd> FOLLOW u10
Command completed successfully
Cmd> TIMELINE
Command completed successfully
Now you are in the timeline
u6 (Tue Sep 30 02:26:45 2025) >> p8

u7 (Tue Sep 30 02:26:51 2025) >> p9
u8 (Tue Sep 30 02:27:16 2025) >> p10

p11
u10 (Tue Sep 30 02:27:26 2025) >> p12
```



```
csce438@cf7e955cca43:~/mp1_skeleton$ ./tsc -h localhost -p 3010 -u u10
Logging Initialized. Client starting...
Login successful as u10

===== TINY SNS CLIENT =====
Command Lists and Format:
FOLLOW <username>
UNFOLLOW <username>
LIST
TIMELINE
=====
Cmd> FOLLOW u7
Command completed successfully
Cmd> FOLLOW u8
Command completed successfully
Cmd> FOLLOW u9
Command completed successfully
Cmd> FOLLOW u6
Command completed successfully
Cmd> TIMELINE
Command completed successfully
Now you are in the timeline
u6 (Tue Sep 30 02:26:45 2025) >> p8

u7 (Tue Sep 30 02:26:51 2025) >> p9

u8 (Tue Sep 30 02:27:16 2025) >> p10

u9 (Tue Sep 30 02:27:20 2025) >> p11

p12
```

Note: All the commands in the testcases mentioned above were executed according to the testcase design document that was given.